
5 Architecture Decisions That Make or Break Embedded Systems

Architecture choices decide everything.

- ▶ Good systems focus on 5 key quality attributes
- ▶ Miss one → problems later
- ▶ Design for all → scalable systems

After working on embedded systems,
I see the same pattern:

Not code. Not tools.

Architecture.

Good systems focus on
5 key quality attributes.

1. Reliability (Robustness)

Can your system handle failures in the real world?

How I design for reliability

- ▶ Abstraction via HAL (Dependency Inversion)
- ▶ Watchdog + Retry patterns
- ▶ State machines → predictable failure behavior

Reliability is not tested later — it's designed early.

0

4

2. Maintainability

Can someone extend your system in 6 months without fear?

How I design for maintainability

- ▶ SOLID principles (SRP, OCP)
- ▶ Modular architecture
- ▶ Strategy / Factory patterns

If one change breaks multiple modules → architecture problem.

05

3. Performance (CPU, Memory, Power)

Can your system meet real-time constraints efficiently?

How I design for performance

- ▶ Static allocation / memory pools
- ▶ RTOS + event-driven design
- ▶ Avoid unnecessary abstractions in critical parts

Most performance issues are architectural—not code-level.

4. Scalability (Extensibility)

Can your system grow without breaking?

How I design for scalability

- ▶ Plugin-based architecture
- ▶ Abstract Factory
- ▶ Publish-Subscribe

Add features without touching existing code.

5. Testability (Debuggability)

Can you verify & debug with confidence?

How I design for testability

- ▶ Dependency injection for mocking
- ▶ Layered testing (unit / integration / system)
- ▶ Logging + trace hooks built in from day one

Let's discuss:

Which is hardest in your system?

Reliability, Maintainability, or Performance?

